Twitter/X 媒体下载器

Java 重写版 数据库结构说明

基于详细设计文档整理

版本：V1.1 日期：2026-10-12

1. 概述

本系统使用 SQLite 作为本地数据库，用于记录下载状态、支持增量同步、断点续传与统计查询。数据库文件默认存放在程序工作目录或配置指定路径（如 data/xmedia.db），程序启动时自动检测并建表。

设计原则：结构简洁、以下载记录为核心、便于扩展用户主页与任务历史；不存储 Cookie 等敏感认证信息。

2. 表关系概览（ER 说明）

核心实体与关系如下：

• download_record（下载记录）— 核心表，每条媒体对应一条记录。

• user_profile（用户主页信息）— 可选，与 download_record 通过 user_rest_id 逻辑关联。

• task_run（任务执行历史）— 可选，记录每次任务运行汇总，与 download_record 通过 task_run_id 可选关联。

• schema_version（结构版本）— 用于数据库迁移管理。

• sync_cursor（增量水位线）— 按用户记录已同步到的最大推文 ID 与最近同步时间，支撑增量同步提前终止分页。

• quota_ledger（调用配额账本）— 按（账号 × 自然日）累计 GraphQL 调用次数，支撑每日配额管理与跨进程续跑。

关系类型：均为逻辑关联（SQLite 可不强制外键），应用层保证一致性。download_record 与 user_profile 为多对一（多条记录对应同一用户）；download_record 与 task_run 为多对一（可选）；download_record 与 sync_cursor 为多对一；quota_ledger 为独立账本表。

3. 表结构详细说明

3.1 download_record（下载记录表）— 核心表

用途：记录每一条已处理媒体的下载结果，作为增量判断与断点续传的依据。

字段名	类型	约束	说明
id	INTEGER	PK, AI	自增主键
tweet_id	TEXT	NOT NULL	推文 ID
media_url	TEXT	NOT NULL	媒体原始 URL（保留展示；不再作为唯一键）
media_type	TEXT	NOT NULL	类型：image / video / gif
local_path	TEXT		本地保存路径（相对或绝对）
user_screen_name	TEXT		所属用户 screen_name（展示用，可变更，非稳定键）
user_rest_id	TEXT		用户 rest_id（稳定键，关联 user_profile.rest_id）
media_key	TEXT	NOT NULL	媒体稳定标识（X media_key；缺失时用 URL 摘要）
variant_bitrate	INTEGER		视频码率变体（可选）
task_type	TEXT		来源任务：USER_MEDIA / TAG / REPLY / TEXT / PROFILE
status	TEXT	NOT NULL	SUCCESS / FAILED（不落 SKIPPED，跳过仅计数不落库）
error_message	TEXT		失败时的错误信息（可截断）
attempt_count	INTEGER		已尝试次数（默认 0），用于重试上限控制
last_attempt_at	TEXT		最近一次尝试时间
file_size	INTEGER		文件大小（字节），可选
tweet_created_at	TEXT		推文发布时间（ISO 或统一格式）
downloaded_at	TEXT		下载完成时间
task_run_id	INTEGER		关联 task_run.id（可选）
created_at	TEXT	NOT NULL	记录插入时间

索引建议：

- UNIQUE(tweet_id, media_key) — 防止同一媒体重复入库，支撑增量判断
- INDEX(user_rest_id)、INDEX(user_screen_name)（展示查询用）
- INDEX(status)
- INDEX(task_type)
- 增量判定只认 status='SUCCESS'；FAILED 记录以 UPSERT 覆盖重试，不参与「已下载」判断

3.2 user_profile（用户主页信息表）— 可选

用途：保存 ProfileTask 获取的用户主页信息，避免重复请求。

字段名	类型	约束	说明

id	INTEGER	PK, AI	自增主键
screen_name	TEXT	NOT NULL	用户 screen_name（可变更，仅展示用）
rest_id	TEXT	UNIQUE, NOT NULL	用户 rest_id（稳定键，关联键）
name	TEXT		显示名称
description	TEXT		简介
avatar_url	TEXT		头像 URL
banner_url	TEXT		Banner URL
avatar_local_path	TEXT		头像本地路径
banner_local_path	TEXT		Banner 本地路径
followers_count	INTEGER		粉丝数（可选）
updated_at	TEXT	NOT NULL	最后更新时间

索引：UNIQUE(rest_id)。与 download_record 通过 user_rest_id = rest_id 逻辑关联；用户改名时按 rest_id 更新本表 screen_name。

3.3 task_run（任务执行历史表）— 可选

用途：记录每次任务运行的汇总信息，便于统计与排查。

字段名	类型	约束	说明
id	INTEGER	PK, AI	自增主键
task_type	TEXT	NOT NULL	任务类型
target	TEXT		目标描述（用户名 / Tag / 链接等）
status	TEXT	NOT NULL	RUNNING / SUCCESS / PARTIAL / FAILED / ABORTED（启动时遗留 RUNNING 自动收敛为 ABORTED）
total_count	INTEGER		计划处理总数
success_count	INTEGER		成功数
failed_count	INTEGER		失败数
skipped_count	INTEGER		跳过数（已存在）
api_call_count	INTEGER		本次运行的 API 调用次数（账号级日累计见 quota_ledger）
started_at	TEXT	NOT NULL	开始时间
finished_at	TEXT		结束时间
duration_ms	INTEGER		耗时（毫秒）
message	TEXT		备注或错误摘要

索引：INDEX(task_type)、INDEX(started_at)。download_record.task_run_id 可指向本表 id。

3.4 schema_version（结构版本表）

用途：记录当前数据库结构版本，便于后续迁移升级。

字段名	类型	约束	说明
version	INTEGER	PK	版本号，从 1 递增
applied_at	TEXT	NOT NULL	应用时间
description	TEXT		变更说明

初始版本建议写入 version=1。后续加表或改字段时递增版本并执行迁移脚本。

3.5 sync_cursor（增量水位线表）

用途：按用户记录已同步到的最大推文 ID 与最近同步时间。增量同步时，若某一页推文全部早于水位线即可提前终止分页，避免每轮翻完整条时间线（直接节省调用配额）。索引：UNIQUE(user_rest_id)。

字段名	类型	约束	说明
id	INTEGER	PK, AI	自增主键
user_rest_id	TEXT	UNIQUE, NOT NULL	用户 rest_id（稳定键）
screen_name	TEXT		用户 screen_name（展示用）
last_max_tweet_id	TEXT		已同步到的最大推文 ID（水位线）
last_synced_at	TEXT		最近一次同步完成时间
last_run_id	INTEGER		关联 task_run.id（可选）
updated_at	TEXT	NOT NULL	最后更新时间

3.6 quota_ledger（调用配额账本表）

用途：按（账号 × 自然日）累计 GraphQL 调用次数，支撑账号级每日配额管理与跨进程续跑；配额耗尽后不得再重试（退避无效），只能换号或次日续跑。索引：UNIQUE(account_name, day)。

字段名	类型	约束	说明
id	INTEGER	PK, AI	自增主键
account_name	TEXT	NOT NULL	账号名（Account.name）
day	TEXT	NOT NULL	自然日（yyyy-MM-dd，本地时区）
call_count	INTEGER	NOT NULL	当日已用调用次数
limit_count	INTEGER		当日配额上限（取自配置，便于诊断）
updated_at	TEXT	NOT NULL	最后更新时间

4. 建表 SQL 参考

以下为 SQLite 建表语句参考（实现时可放在 Database 初始化逻辑中）：

连接参数（每个连接打开后执行）：PRAGMA journal_mode = WAL; PRAGMA busy_timeout

= 5000; PRAGMA synchronous = NORMAL; PRAGMA foreign_keys = ON;

写库约定：并发下载线程不直接写库，结果经单写线程队列串行落库，避免 SQLITE_BUSY 被误判为下载失败。

```
-- download_record
CREATE TABLE IF NOT EXISTS download_record (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  tweet_id TEXT NOT NULL,
  media_url TEXT NOT NULL,
  media_key TEXT NOT NULL,
  media_type TEXT NOT NULL,
  variant_bitrate INTEGER,
  local_path TEXT,
  user_screen_name TEXT,
  user_rest_id TEXT,
  task_type TEXT,
  status TEXT NOT NULL,
  error_message TEXT,
  attempt_count INTEGER DEFAULT 0,
  last_attempt_at TEXT,
  file_size INTEGER,
  tweet_created_at TEXT,
  downloaded_at TEXT,
  task_run_id INTEGER,
  created_at TEXT NOT NULL,
  UNIQUE(tweet_id, media_key)
);
CREATE INDEX IF NOT EXISTS idx_dr_user ON download_record(user_screen_name);
CREATE INDEX IF NOT EXISTS idx_dr_status ON download_record(status);
CREATE INDEX IF NOT EXISTS idx_dr_task_type ON download_record(task_type);

CREATE INDEX IF NOT EXISTS idx_dr_user_rest ON download_record(user_rest_id);

-- user_profile
CREATE TABLE IF NOT EXISTS user_profile (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  screen_name TEXT NOT NULL,
  rest_id TEXT NOT NULL UNIQUE, name TEXT, description TEXT,
  avatar_url TEXT, banner_url TEXT,
  avatar_local_path TEXT, banner_local_path TEXT,
  followers_count INTEGER, updated_at TEXT NOT NULL
);

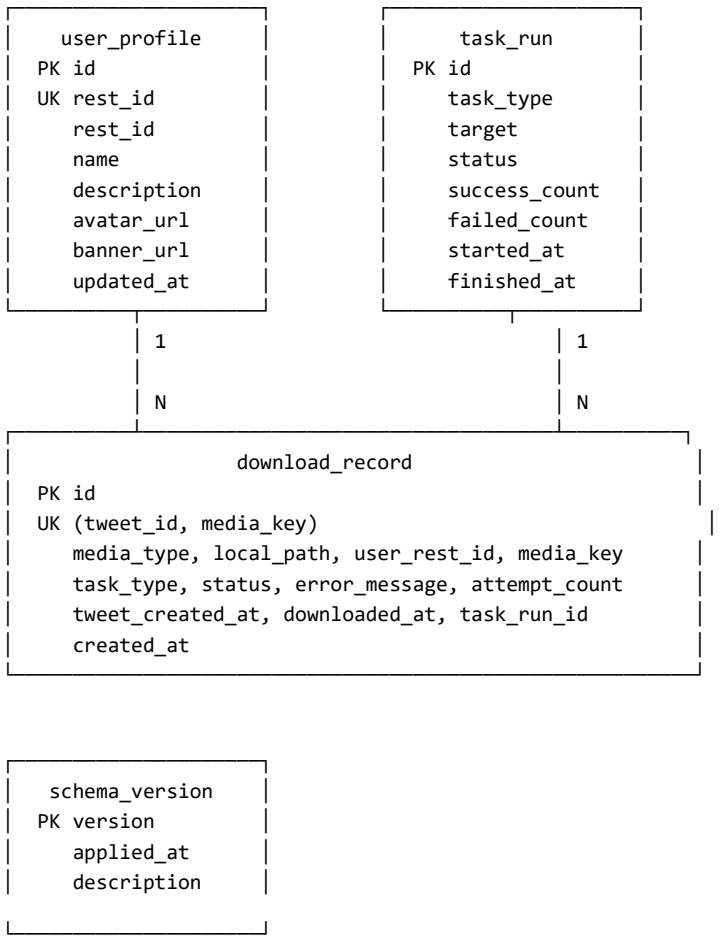
-- task_run
CREATE TABLE IF NOT EXISTS task_run (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  task_type TEXT NOT NULL, target TEXT, status TEXT NOT NULL,
  total_count INTEGER, success_count INTEGER, failed_count INTEGER, skipped_count INTEGER,
  api_call_count INTEGER, started_at TEXT NOT NULL, finished_at TEXT,
  duration_ms INTEGER, message TEXT
);

-- sync_cursor
CREATE TABLE IF NOT EXISTS sync_cursor (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_rest_id TEXT NOT NULL UNIQUE, screen_name TEXT,
  last_max_tweet_id TEXT, last_synced_at TEXT, last_run_id INTEGER,
  updated_at TEXT NOT NULL
);
-- quota_ledger
CREATE TABLE IF NOT EXISTS quota_ledger (
```

```
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  account_name TEXT NOT NULL, day TEXT NOT NULL,
  call_count INTEGER NOT NULL DEFAULT 0, limit_count INTEGER,
  updated_at TEXT NOT NULL,
  UNIQUE(account_name, day)
);
-- schema_version
CREATE TABLE IF NOT EXISTS schema_version (
  version INTEGER PRIMARY KEY, applied_at TEXT NOT NULL, description TEXT
);
```

5. 数据库结构图（文字版）

可用下列关系绘制 ER 图：



注：另有 sync_cursor（按用户水位线）与 quota_ledger（账号 × 日 调用配额账本）两张辅助表未画入上图，二者均为独立表、无强外键。

6. 典型查询场景

• 增量判断：SELECT 1 FROM download_record WHERE tweet_id=? AND media_key=? AND status='SUCCESS' LIMIT 1

• 某用户已下载列表：SELECT * FROM download_record WHERE user_rest_id=? AND status='SUCCESS'

• 失败重试列表：SELECT * FROM download_record WHERE status='FAILED'

• 清理某用户缓存：DELETE FROM download_record WHERE user_rest_id=?（配合三档清理：仅记录 / 记录+文件 / 含空目录）

• 任务历史：SELECT * FROM task_run ORDER BY started_at DESC LIMIT 20

• 失败重试（UPSERT 覆盖）：INSERT INTO download_record(tweet_id, media_key, media_type, status, attempt_count, created_at) VALUES(?,?,?, 'SUCCESS', 1, ?) ON CONFLICT(tweet_id, media_key) DO UPDATE SET status=excluded.status, local_path=excluded.local_path, attempt_count=download_record.attempt_count+1;

• 读取水位线：SELECT last_max_tweet_id FROM sync_cursor WHERE user_rest_id=?

• 今日配额已用：SELECT call_count, limit_count FROM quota_ledger WHERE account_name=? AND day=?

7. 补充说明

• 第一阶段实现 download_record + schema_version + sync_cursor + quota_ledger（后两者是增量同步与每日配额的必要支撑），user_profile 与 task_run 在第二/三阶段按需启用。

• 不在数据库中存储 Cookie、auth_token 等敏感信息。

• 时间字段统一使用 ISO-8601 字符串或约定格式，便于跨平台比较。

• 外键可为逻辑关联，若开启 SQLite 外键，需在连接时执行 PRAGMA foreign_keys = ON；并发场景另需 journal_mode = WAL 与 busy_timeout（见第 4 节），写入经单写队列串行化。

• 后续若增加“文本推文归档”等需求，可新增 text_record 表，结构与 download_record 类似。

8. 文档效力与修订记录

文档效力顺序：本套文档共五份（需求分析、详细设计、类图说明、时序图与关键流程图、数据库结构说明）。效力顺序为：需求分析 > 详细设计 > 类图说明 / 时序图与关键流程图 / 数据库结构说明（三者同级，冲突时以详细设计为准）。出现冲突时，以顺序在前的文档为准。

修订记录：

版本	日期	说明	备注
V1.1	2026-10-12	依据评审意见整改：H1 增量判定改认 status='SUCCESS' 且 FAILED 允许 UPSERT 覆盖（§3.1、§6）；H3 增连接 PRAGMA 与单写队列约定（§4、§7）；H5/S6 增 user_rest_id、media_key、variant_bitrate，user_profile 改用 rest_id 为稳定键（§3.1、§3.2、§5）；S3 删除 SKIPPED 枚举（§3.1）；S4 增 attempt_count / last_attempt_at（§3.1）；S5 新增 sync_cursor 表（§3.5）；H2 新增 quota_ledger 表（§3.6）；S7 task_run 增 ABORTED 与启动收敛说明（§3.3）；S8 清理缓存三档（§6）；S14 增文档效力顺序与本表	评审后首版

— 文档结束 —